

Vorabfragen

- Sie wollen eine Tabelle `data` mit ganzzahligen Schlüsseln speichern und direkt `data[key]` als Speicherort verwenden. Es sollen aber nur zwei Einträge abgelegt werden, mit den Schlüsseln 0 und 9.999.999.999. Was ist an diesem Vorgehen problematisch? Welche Idee hätten Sie, um trotzdem schnell auf die Einträge zuzugreifen, ohne ein riesiges Array anzulegen?
- Ein Array hat 10 Plätze, und der Index eines Schlüssels wird als $\text{key} \% 10$ berechnet. Nacheinander werden die Paare (5, "a"), (15, "b") und (25, "c") eingefügt. Was passiert beim zweiten und dritten Einfügen? Wie würden Sie die Datenstruktur anpassen, damit kein Paar verloren geht?
- Sie sollen Strings (z. B. E-Mail-Adressen) auf einen Array-Index abbilden. Drei Vorschläge:
 - (a) die Länge des Strings
 - (b) die Summe der Zeichencodes aller Buchstaben
 - (c) eine Zufallszahl (`random.nextInt()`)
- Welche Eigenschaften sollte eine gute Hashfunktion haben? Welcher Vorschlag erfüllt diese am besten, und was geht bei den anderen schief?



Algorithmik – WS 26/27

Gelesen von Prof. Dr. Daniel Gaida

Übungsaufgabe – Diskutieren Sie gerne mit Nachbar:in

Stellen Sie sich vor, Sie arbeiten für ein großes soziales Netzwerk und müssen eine Funktion entwickeln, die schnell überprüft, ob für eine bestimmte E-Mail-Adresse ein Nutzer bereits registriert ist. Welchen abstrakten Datentyp würden Sie wählen und welche Datenstruktur würden Sie für die Umsetzung wählen?

E-Mailadresse (Schlüssel)	User (Wert)
daniel@th-koeln.de	Daniel
max@mustermann.de	Max

Tabellenimplementierungen

Worst Case Laufzeit

	ArrayDictionary	LinkedDictionary
put(key, value)	$O(n)$	$O(n)$
get(key)	$O(n)$	$O(n)$
containsKey(key)	$O(n)$	$O(n)$
remove(key)	$O(n)$	$O(n)$
size()	$O(1)$	$O(1)$

Beide Implementierungen sind nicht so toll.

Deshalb lernen wir bald bessere Implementierungen kennen.

- Hashtabelle, Bäume, ...

Lernraum II: Höhere Abstrakte Datentypen und Datenstrukturen

- Abstrakte Datentypen
 - Prioritätswarteschlange
- Datenstrukturen
 - **Hashtabelle**
 - Binäre Suchbäume
 - AVL-Baum
 - Heap

Lernziele von heute und Fragen zur Überprüfung der Lernziele

Die Studierenden können eine Tabelle für beliebige Daten anlegen, die eine konstante Laufzeit für Lese-/Schreiboperationen hat,

- indem sie Hashing nutzen,
- dabei Hashfunktionen und Techniken zum Auflösen von Hash-Kollisionen anwenden
- und die Tabelle (aka Hashtabelle) rechtzeitig vergrößern,

um später beliebige Daten in konstanter Laufzeit abspeichern und wieder abrufen zu können.

s. Übungsblatt 4

Gliederung der Vorlesung

Tabelle ADT

Wiederholung

1

ArrayDictionary
Tabelle

2

**Direkt adressierbare
Tabelle**

SCHNELLER: Direkt
Element adressieren,
nur Integer Schlüssel

3

Hashtabelle

FLEXIBLER: Die
Hashfunktion
unterstützt jede Art
von Schlüssel

4

**Hashing mit
Verkettung**

IHR BESTER FREUND:
Löst Hash-Kollisionen,
ist aber trotzdem
schnell!

Direkt adressierbare Tabelle (DirectAccessMap)

Problemvereinfachung: nur Unterstützung ganzzahliger Schlüssel

Datenstruktur: Array; wg. $\Theta(1)$ -Laufzeit für Zugriff auf ein Element (`data[key]`) oder die Aktualisierung eines Elements (`data[key] = ...`).

Idee: Speichern des Schlüssel-Wert-Paars unter `data[key]`

- Konstante Laufzeit für `put`, `containsKey` und `get` - wir können direkt zu `data[key]` springen, um den Wert zu schreiben/auszulesen.

DirectAccessMap<Integer, V>

state

`data[]`
`size`

behavior

`put(key, value)` put key-value-pair at given key/index
`get(key)` get value at given key/index
`containsKey(key)` if `data[]` null at key/index, return false, return true otherwise
`remove(key)` nullify value at key/index
`size()` return count of key-value-pairs

```
put(3, "Hans");
get(3);
```

indices	0	1	2	3	4	5	6	7	8	9
data				(3, "Hans")						

Direkt adressierbare Tabelle

Implementierung

```
public void put(int key, V value) {
    this.data[key] = (key, value);
}

public boolean containsKey(int key) {
    return this.data[key] != null;
}

public V get(int key) {
    return this.data[key][1];
}

public void remove(int key) {
    this.data[key] = null;
}
```

DirectAccessMap<Integer, V>

state

data[]
size

behavior

put(key, value) put key-value-pair at given key/index
get(key) get value at given key/index
containsKey(key) if data[] null at key/index, return false, return true otherwise
remove(key) nullify value at key/index
size() return count of key-value-pairs

Operation		Array w/ indices as keys
put(key, value)	best	$\Theta(1)$
	worst	$\Theta(1)$
get(key)	best	$\Theta(1)$
	worst	$\Theta(1)$
containsKey(key)	best	$\Theta(1)$
	worst	$\Theta(1)$

Direkt adressierbare Tabelle: Vor- und Nachteile

Vorteile

- Sehr schnell:
 - Laufzeit von $\Theta(1)$ für alle Operationen

Nachteile

- Speicherbedarf
 - Tabelle mit den beiden Schlüsseln: 0 und 9999999999: Unser derzeitiges Konzept würde den ganzen Array-Speicherplatz dazwischen verschwenden.
- Nur ganzzahlige Schlüssel
 - Starke Einschränkung; Aber, die schnelle Ermittlung des Array Indizes aus dem Schlüssel ist sehr wertvoll, da Index-basierte Array Operationen in $\Theta(1)$ sind (keine Schleifen benötigt wie bei ArrayDictionary/LinkedDictionary).
- Verknüpfung von Schlüssel und Index gilt für alle Tabellen, die wir heute besprechen.

Direkt adressierbare Tabelle

Können die ganzzahligen Schlüssel beliebig groß sein?

Idee bisher:

- Erstellen eines RIESIGEN Arrays in dem alle möglichen ganzzahligen Schlüssel 1-zu-1 auf einen Index abgebildet werden

Probleme:

- Wir können kein Array erstellen, das **alle** Zahlen enthält (unendlich)
- Sehr verschwenderisch bezgl. Speicherbedarf

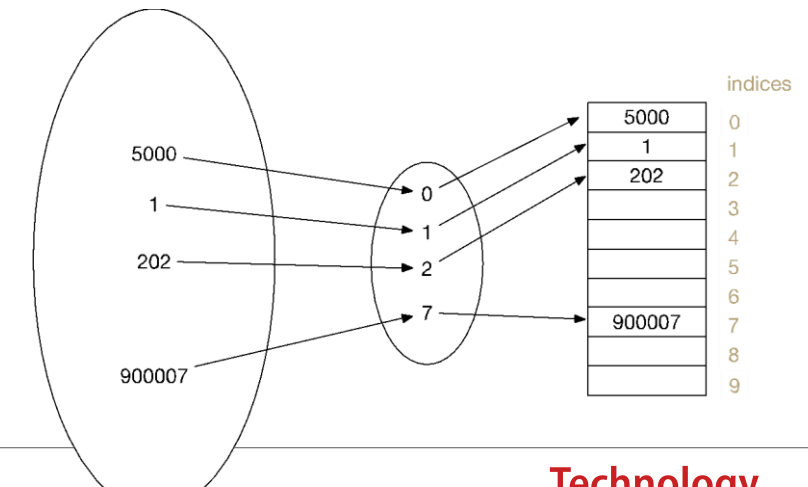
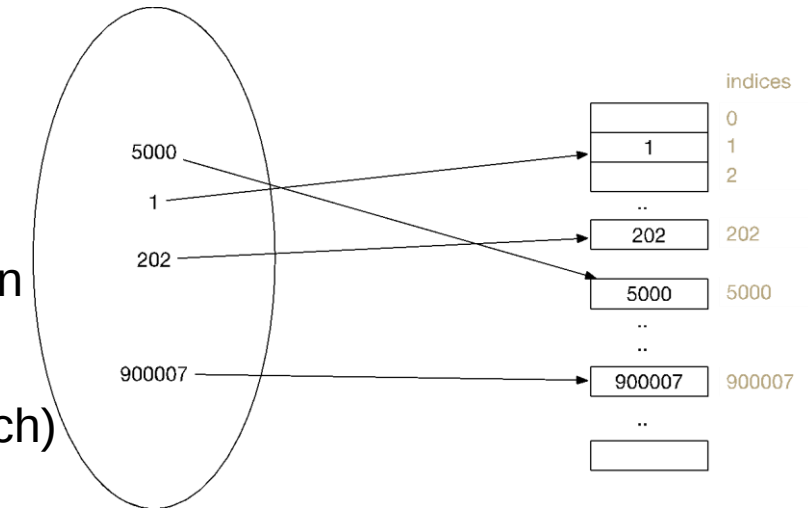
Neue Idee:

- Ein kleineres Array erstellen, aber eine Möglichkeit schaffen, gegebene Integer-Schlüssel auf verfügbare Indizes abzubilden. Viel effizienter bezgl. Speicherbedarf.

Problem:

- Wie können wir eine gute Abbildung bestimmen?

In dieser Abb. sind nur die Schlüssel abgebildet, Werte hinzu denken



Hashfunktionen: Umwandlung von Daten in einen Integer

Definition der Hashfunktion

Eine Hashfunktion ist eine Funktion, die verwendet werden kann, um Daten beliebiger Größe auf Ganzzahlen mit einem begrenzten Wertebereich abzubilden.

In unserem Fall:

- Abbilden eines Integer Schlüssels auf einen gültigen Index im Array.
Bsp.: Array der Größe 10; Schlüssel beträgt 500: Hashfunktion bildet Schlüssel auf eine Zahl zwischen 0 und 9 ab.

Eine einfache Lösung:

- Hashfunktion: Schlüssel % (Modulo; Restwert-Division) durch Arraygröße.
- Beispiel: Schlüssel ist 500; Arraygröße ist 10: Da $500 \% 10 = 0$, würden wir das Schlüssel-Wert-Paar bei Index 0 einfügen.

Ganzzahliger Rest mit % „mod“ (Modulo-Operator)

Der %-Operator berechnet den Rest einer ganzzahligen Division.

$$14 \% 4 = 2$$

$$218 \% 5 = 3$$

Äquivalent: $a \% b$ (für $a, b > 0$):

```
while(a > b - 1)
    a -= b;
return a;
```

Anwendungen des %-Operators:

- Ermitteln der letzten Ziffer einer Zahl: $230857 \% 10 = 7$
- Feststellung, ob eine Zahl ungerade ist: $7 \% 2 = 1$, $42 \% 2 = 0$
- Ganzzahlen auf einen bestimmten Bereich beschränken:
 - $8 \% 12 = 8$, $18 \% 12 = 6$



Schlüssel auf Indizes
innerhalb des Arrays
beschränken

Erste Hashfunktion: % Arraygröße

indices	0	1	2	3	4	5	6	7	8	9
elements	(0, "foo")	(11, "biz")				(5, "bar")			(18, "bop")	

```

put(0, "foo");    0 % 10 = 0
put(5, "bar");    5 % 10 = 5
put(11, "biz");   11 % 10 = 1
put(18, "bop");   18 % 10 = 8
    
```

Unsere erste Hashtabelle

```
public void put(int key, V value) {
    this.data[hashToValidIndex(key)] = (key, value);
}

public V get(int key) {
    return this.data[hashToValidIndex(key)][1];
}

private int hashToValidIndex(int k) {
    return k % this.data.length;
}
```

Hinweis: % ist nur ein mathematischer Operator wie +, -, /, *, hat also eine konstante Laufzeit.

SimpleHashMap<Integer>

state

data[]
size

behavior

put(key, value) mod key by table size,
put key-value-pair at result
get(key) mod key by table size, get
value at result
containsKey(key) mod key by table
size, return data[result] != null
remove(key) mod key by table size,
nullify element at result
size return count of key-value-pairs

Operation		Array w/ indices as keys
put(key,value)	best	$\Theta(1)$
	worst	$\Theta(1)$
get(key)	best	$\Theta(1)$
	worst	$\Theta(1)$
containsKey(key)	best	$\Theta(1)$
	worst	$\Theta(1)$

Fragen?

Dinge, über die wir gesprochen haben:

- Wiederholung von ArrayDictionary und LinkedDictionary
- Direkt adressierbare Tabelle (DirectAccessMap)
- % als Hashfunktion und SimpleHashMap

Erste Hashfunktion: % Arraygröße

indices	0	1	2	3	4	5	6	7	8	9
elements	(20, ":(")	(11, "biz")				(5, "bar")			(18, "bop")	

```

put(0, "foo");    0 % 10 = 0
put(5, "bar");    5 % 10 = 5
put(11, "biz");   11 % 10 = 1
put(18, "bop");   18 % 10 = 8
put(20, ":(");    20 % 10 = 0
    
```



Kollision!

Problem des Hashings: Kollisionen

Kollision:

- mehrere unterschiedliche Schlüssel werden auf denselben Index des Arrays abgebildet

Zwei Fragen:

1. Wenn wir eine Kollision haben, wie lösen wir sie auf?
2. Wie können wir die Anzahl der Kollisionen minimieren?
 - Da das Auflösen von Kollisionen Zeit benötigt, gilt je weniger Kollisionen, desto besser die Laufzeit!

Strategien zum Umgang mit Hash-Kollisionen

Es gibt mehrere Strategien. In diesem Kurs werden wir die folgenden behandeln:

1. Hashing mit Verkettung
2. Offene Adressierung
 - Lineare Sondierung
 - Quadratische Sondierung
 - Doppeltes Hashing

Hashing mit Verkettung

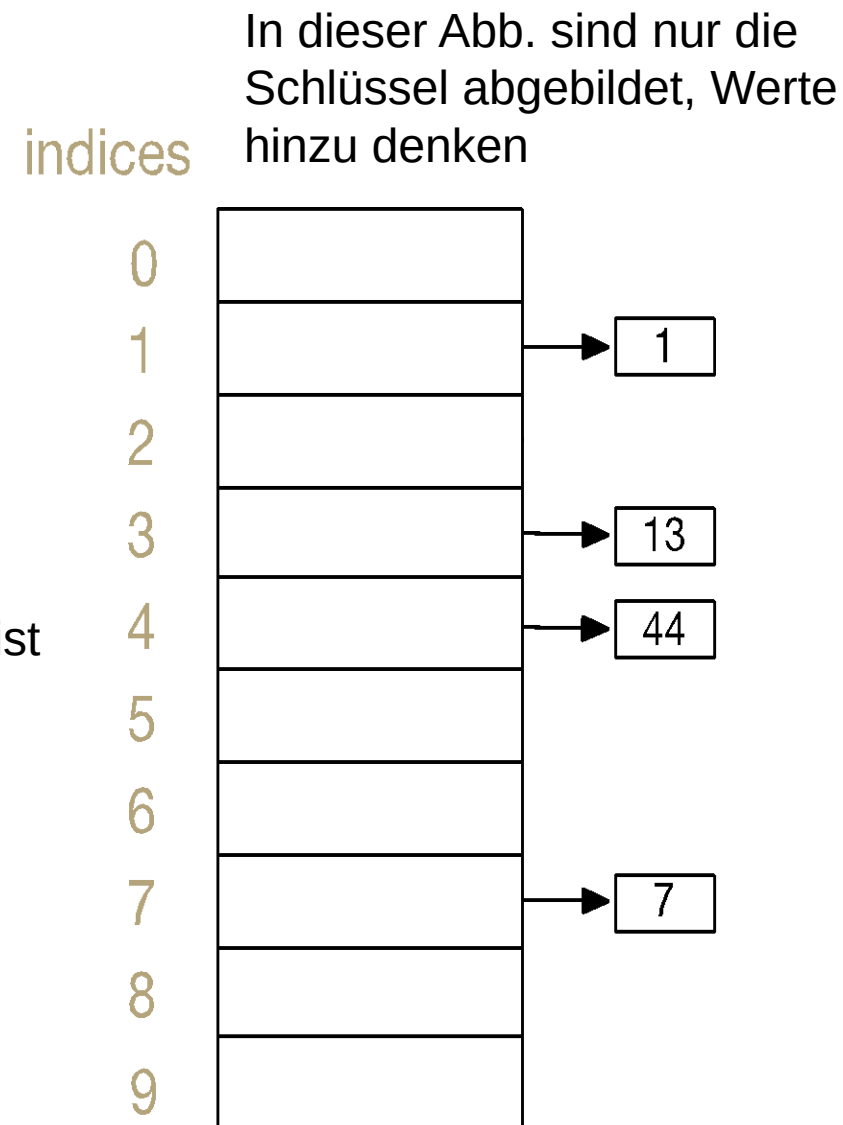
Jeder Index in unserem Array stellt einen „Slot“ dar.

Wenn ein Schlüssel-Wert-Paar (k,v) auf den Index h hasht:

- Wenn der Slot bei Index h leer ist:
 - eine neue verkettete Liste erstellen, die (k,v) enthält
- Wenn der Slot bei Index h bereits eine verkettete Liste ist:
 - füge (k,v) hinzu, wenn der Schlüssel noch nicht vorhanden ist

Mit anderen Worten:

- Wenn mehrere Schlüssel-Wert-Paare auf denselben Index verweisen, fügen wir sie einfach alle in denselben Slot ein. Häufig wird eine Datenstruktur gewählt, die einer verketteten Liste ähnelt.



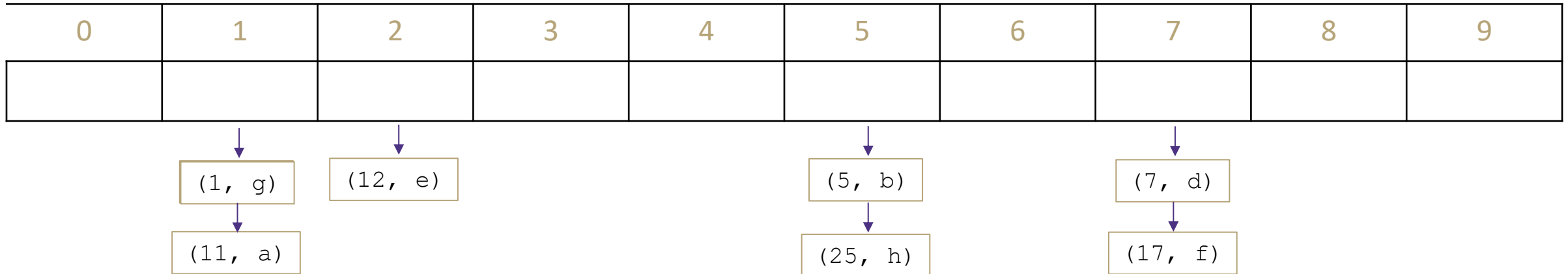
Hashing mit Verkettung - Übung

Befüllen Sie die Tabelle `data` mit einer Größe von 10 durch Hashing mit Verkettung. Die Slots sind durch eine LinkedList implementiert, an deren Ende Sie neue Schlüssel-Wert-Paare anhängen.

Fügen Sie die folgenden Schlüssel-Wert-Paare ein (Hashfunktion: `key % data.length`).

Wie sieht die Hashtabelle aus?

(1, a), (5,b), (11,a), (7,d), (12,e), (17,f), (1,g), (25,h)



Hashing mit Verkettung

Zur Erinnerung: die Implementierungen von put/get/containsKey sind alle sehr ähnlich und die Laufzeit hat fast immer die selbe Wachstumsordnung

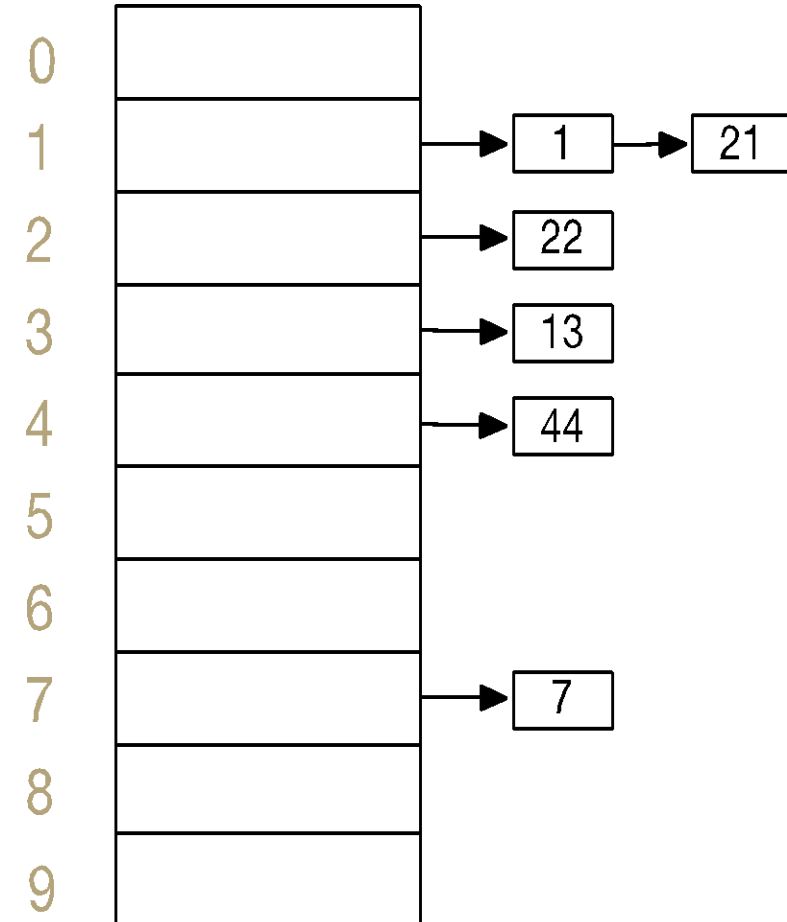
```
// some pseudocode
public boolean containsKey(int key) {
    int slotIndex = key % data.length;
    loop through data[slotIndex]
        return true if we find the key in
            data[slotIndex]
    return false if we get to here (didn't find it)
}
```

Laufzeitanalyse

Gibt es verschiedene mögliche Zustände für unsere Hashtabelle, die diesen Code langsamer/schneller machen, wenn bereits n Schlüssel-Wert-Paare gespeichert sind?

Ja! Wenn wir viele Schleifendurchgänge durchführen müssten, um den Schlüssel im Slot zu finden, würde unser Code langsamer laufen.

indices

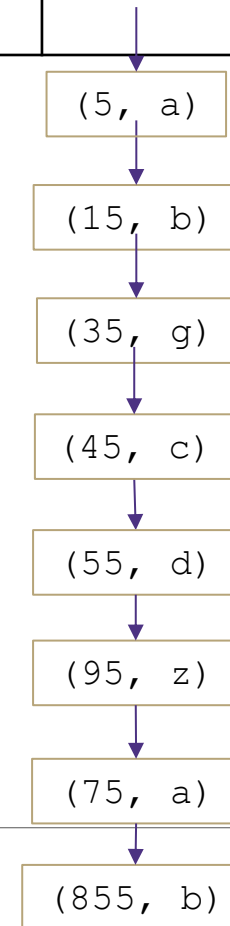


Hashing mit Verkettung

Worst Case Zustand

0	1	2	3	4	5	6	7	8	9

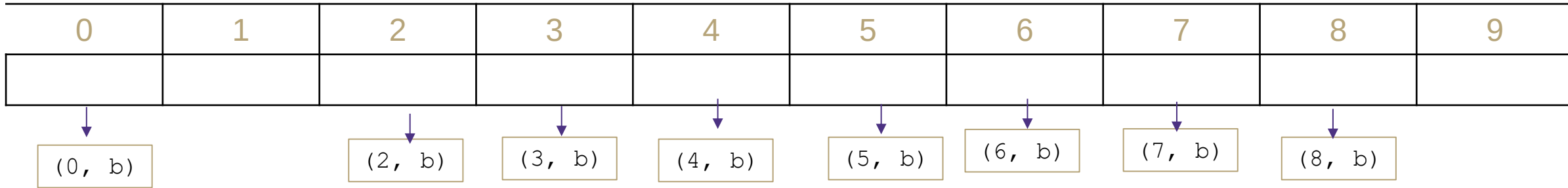
- Es ist möglich, dass alle Schlüssel-Wert-Paare auf den gleichen Slot ghasht werden
- In diesem Worst Case Fall hat `containsKey` eine Laufzeit von $\Theta(n)$
- Beispiel:
 - `containsKey(555)`
- Gehe zu Index 5 und prüfe für alle n Schlüssel-Wert-Paare in dem Slot, ob sich der Schlüssel 555 in dem Slot befindet



Quelle: Data Structures and Algorithms (CSE 373),
University of Washington, Spring 2023

CSE 373 - ROBBIE WEBER + HANNAH TANG

Hashing mit Verkettung: Best Case



Es ist möglich, dass alle Schlüssel-Wert-Paare ziemlich gleichmäßig auf die Slots verteilt sind.

Gegenteil zur letzten Folie: Bei Slots mit nur 0 oder 1 Schlüssel-Wert-Paar, wird `containsKey` für diese Slots eine konstante Laufzeit haben.

Das Befolgen von Best Practices erhöht die Wahrscheinlichkeit, dass wir diesen Best Case Zustand erreichen.

Hashing mit Verkettung: Situation in der Praxis

Best Case und Worst Case beschreiben die Verteilung der Schlüssel-Wert-Paare und nicht mehr die Position des Schlüssel-Wert-Paares in dem Array

Die Laufzeiten „in der Praxis“ gehen von einer

- gleichmäßigen Verteilung der Schlüssel-Wert-Paare innerhalb des Arrays aus und
 - Hashfunktion
- dem Befolgen von Best Practices, um eine geringe durchschnittliche Verkettungslänge zu gewährleisten.
- Rechtzeitige Größenänderung der Hashtabelle

Operation		Array w/ indices as keys
put(key,value)	best	$\Theta(1)$
	In der Praxis	Wunsch: $\Theta(1)$
	worst	$\Theta(n)$
get(key)	best	$\Theta(1)$
	In der Praxis	Wunsch: $\Theta(1)$
	worst	$\Theta(n)$
remove(key)	best	$\Theta(1)$
	In der Praxis	Wunsch: $\Theta(1)$
	worst	$\Theta(n)$

Ändern der Größe der Hashtabelle

Was ist mit der Größenänderung?

- Bei Datenstrukturen wie ArrayDictionary, ArrayList oder ArrayStack mussten wir größere Arrays nutzen, wenn das Array voll war, weil wir nicht mehr Daten speichern konnten! Bei Hashing mit Verkettung ist das anders: wir sind nie gezwungen, ein größeres Array zu nutzen, da Slots eine flexible Größe haben.
- Allerdings möchten wir trotzdem „ab und zu“ ein größeres Array nutzen, um sicherzustellen, dass die durchschnittliche/erwartete Länge der verketteten Liste in jedem Slot eine kleine Zahl ist.
- Beispiel: Array mit 10 Slots enthält 100 Schlüssel-Wert-Paare:
- Unter der Annahme des „In der Praxis“ Falls (nicht der Worst Case), erwarten wir, dass im Durchschnitt jeder der 10 Slots etwa 10 Schlüssel-Wert-Paare enthält.
- Was passiert, wenn wir die Größe des Arrays beibehalten, aber 100 weitere Schlüssel-Wert-Paare hinzufügen?
- Jeder Slot erhält etwa 10 weitere Schlüssel-Wert-Paare und die Laufzeit wird immer schlechter.

Ändern der Größe der Hashtabelle

Belegungsfaktor Lambda

Die Laufzeit der Operationen korreliert mit der durchschnittlichen Belegung der Slots:

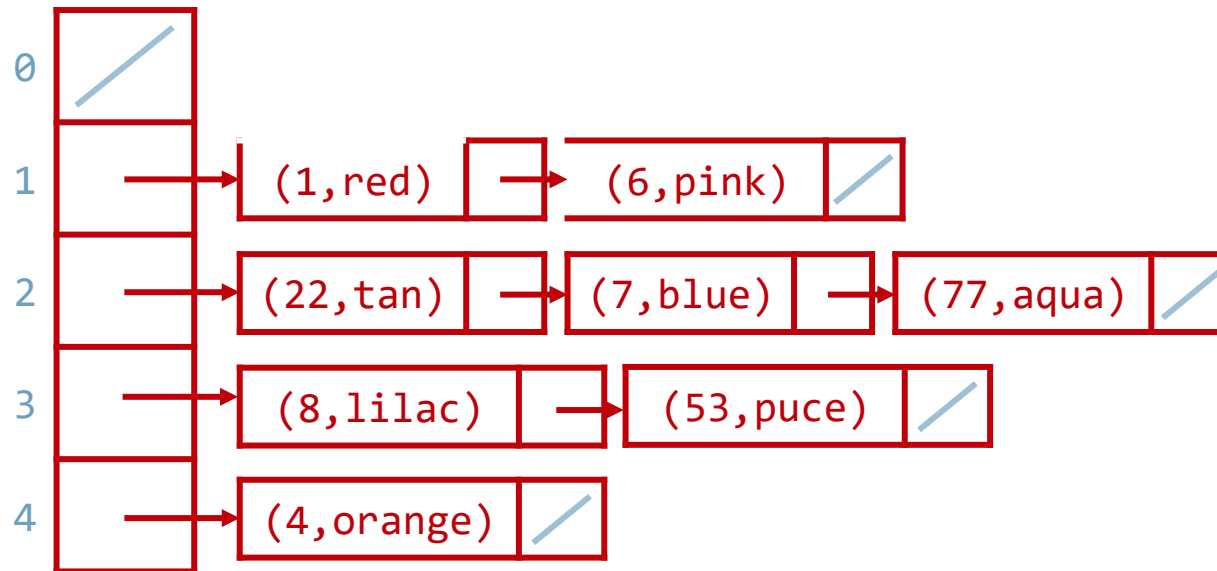
- Anzahl der Schlüssel-Wert-Paare / Anzahl der Slots
- Beispiel: $\lambda = 8 / 5 = 1,6$

Belegungsfaktor λ

n : Zahl an Schlüssel-Wert-Paaren

c : Größe des Arrays (# Slots)

$$\lambda = \frac{n}{c}$$



Ändern der Größe der Hashtabelle

Belegungsfaktor Lambda

- Wenn die Arraygröße fixiert ist, dann hat `containsKey` eine Laufzeit der Größenordnung n
- Wenn Sie ein größeres Array nutzen (wenn weitere Schlüssel-Wert-Paare gespeichert werden) und Sie die Schlüssel-Wert-Paare gleichmäßig auf die Slots verteilen, können Sie Ihre Laufzeit niedrig halten.

Wenn Sie ein größeres Array nutzen, wenn λ einen konstanten Wert wie 1 erreicht:

- Wir können 1 Paar pro Slot erwarten: konstante Laufzeit!
- Wenn wir die Größe der Hashtabelle jedes Mal verdoppeln, wird der teure Größenänderungsvorgang immer seltener
- Best Practice: Arraygröße verdoppeln und nächstgrößere Primzahl wählen

Belegungsfaktor λ

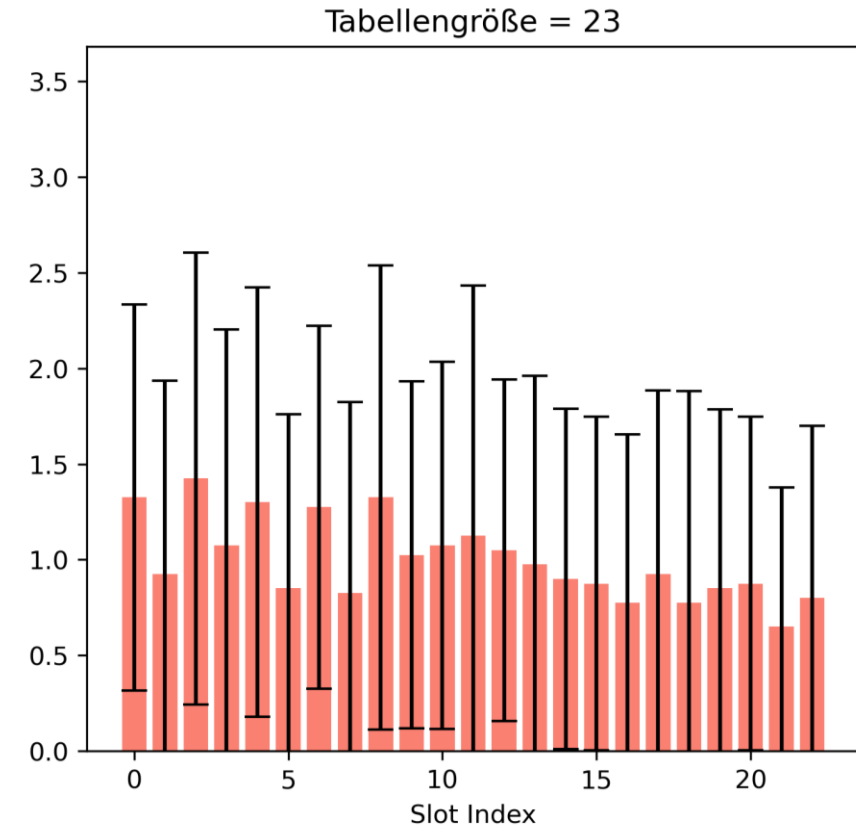
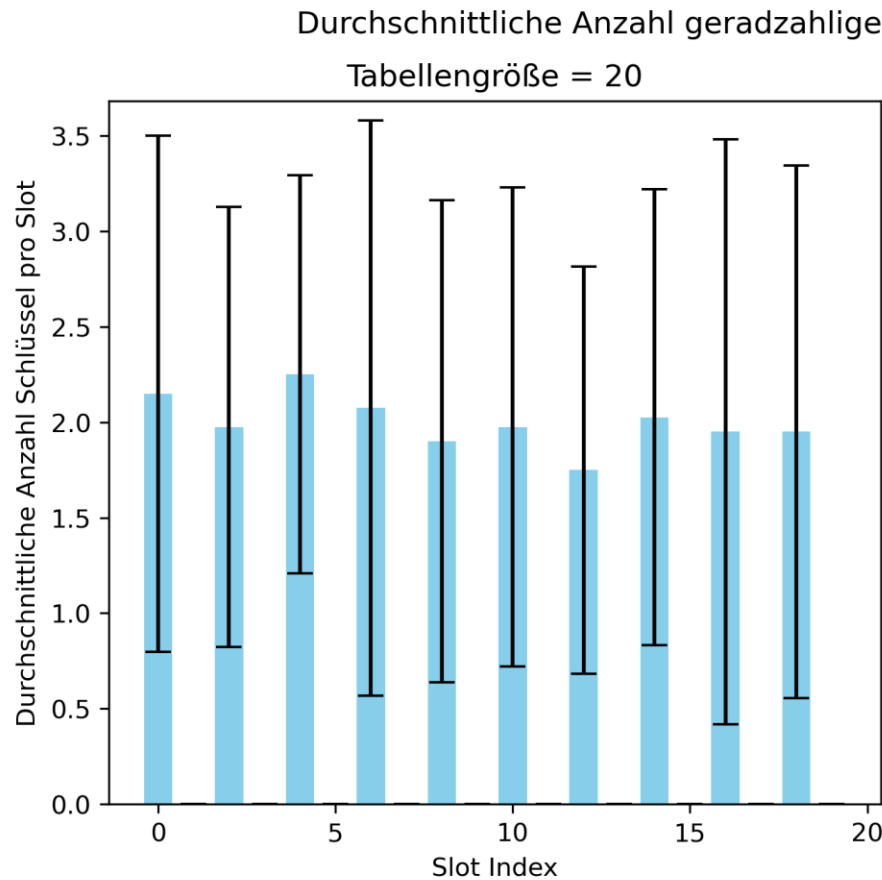
n : Zahl an Schlüssel-Wert-Paaren

c : Größe des Arrays (# Slots)

$$\lambda = \frac{n}{c}$$

Ändern der Größe der Hashtabelle

Warum sollte die Größe einer Hashtabelle eine Primzahl sein?



Ändern der Größe der Hashtabelle

Belegungsfaktor Lambda

Operation		Array w/ indices as keys
put(key, value)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$
get(key)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$
remove(key)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$

Fall „in der Praxis“ hängt von mittlerer Anzahl von Schlüssel-Wert-Paaren pro Slot ab

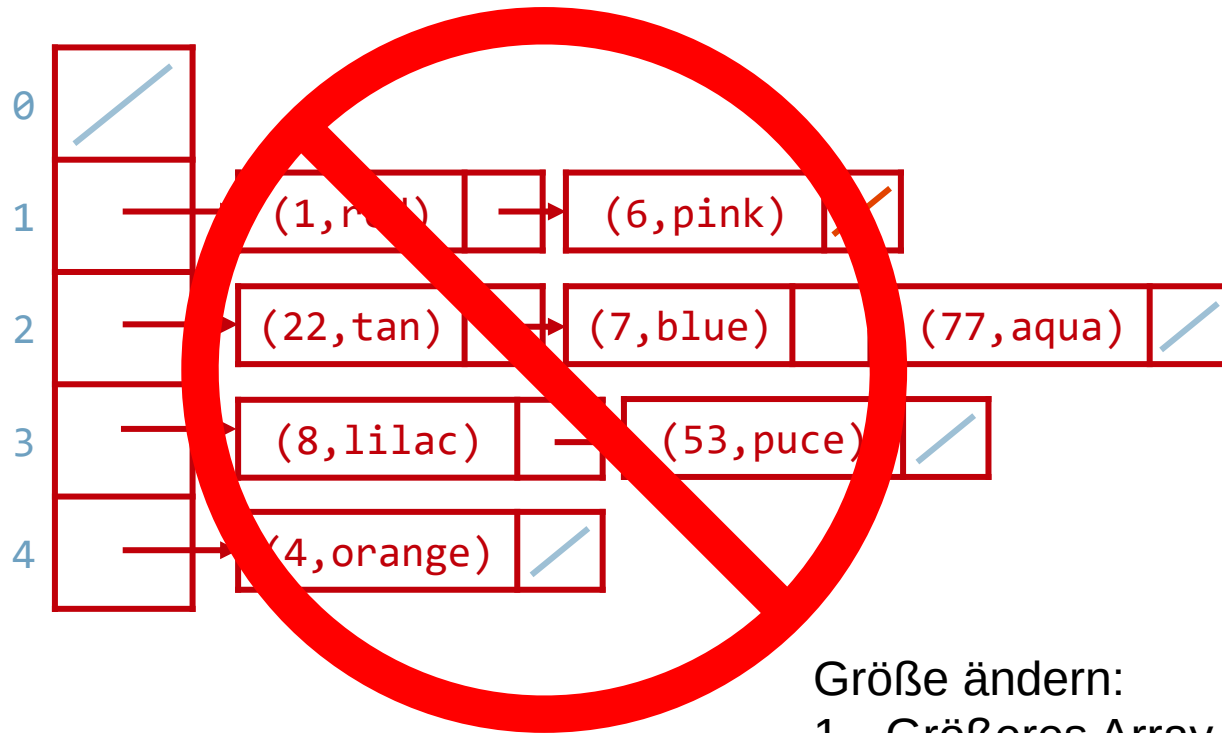
Belegungsfaktor λ

n : Zahl an Schlüssel-Wert-Paaren

c : Größe des Arrays (# Slots)

$$\lambda = \frac{n}{c}$$

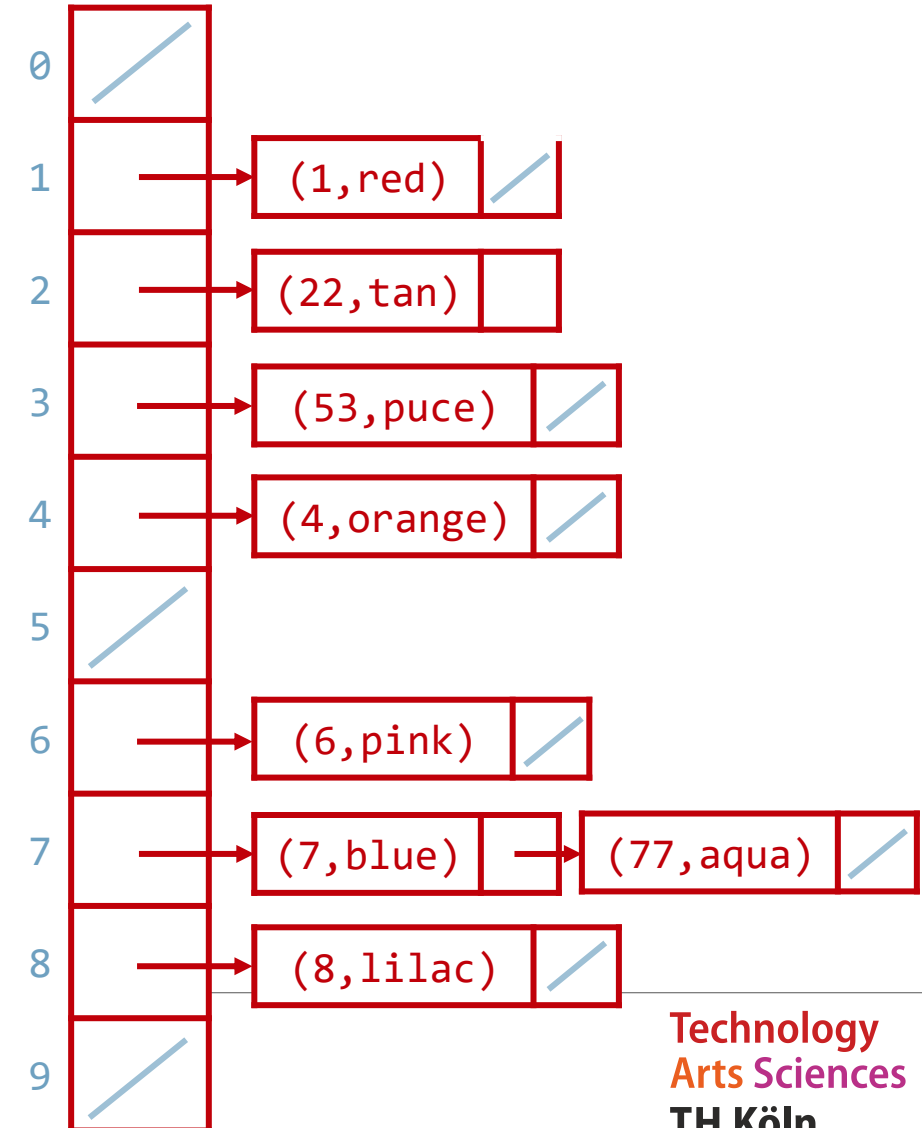
Ändern der Größe der Hashtabelle



Schlüssel-Wert-Paare
NICHT einfach im
alten Slot eintragen

Größe ändern:

1. Größeres Array erstellen
2. Jedes Paar aus der alten Hashtabelle neu verteilen!
Berechne Position neu,
indem Sie Schlüssel %
neuen Größe rechnen



Fragen?

- Hashing mit Verkettung
- Belegungsfaktor

- Übungsblatt 4: Aufgabe 1 a

Hashfunktion für Schlüssel, die keine Ganzzahlen sind

Definition der Hashfunktion

Eine Hashfunktion ist eine Funktion, die verwendet werden kann, um Daten beliebiger Größe auf Ganzzahlen mit einem begrenzten Wertebereich abzubilden.

Wir werden eine Hashfunktion definieren, die Strings auf Integer abbilden kann.

Wie sehen gute Hashfunktionen aus?

Die Hashfunktion einer Hashtabelle wird sehr oft aufgerufen:

- Beim Einfügen von Schlüssel-Wert-Paaren in die Hashtabelle und bei get(key)
- Bei der Überprüfung, ob ein Schlüssel bereits in der Hashtabelle enthalten ist
- Bei der Größenänderung und Umverteilung aller Schlüssel-Wert-Paare in eine neue Hashtabelle

Aus diesem Grund ist es wichtig, eine „gute“ Hashfunktion zu haben. Eine gute Hashfunktion ist:

- Deterministisch - die gleiche Eingabe sollte die gleiche Ausgabe erzeugen
- Effizient - sie sollte eine vertretbare Laufzeit haben
- Gleichmäßig - die Schlüssel sollten „gleichmäßig“ über die Tabelle verteilt werden (→ Best Case)

```
public int hashFn(String key) {
    return random.nextInt();
}
```

NICHT deterministisch

```
public int hashFn(String key) {
    if (key.length() % 2 == 0)
        return 17;
    else
        return 43;
}
```

NICHT gleichmäßig

```
public int hashFn(String key) {
    int retVal = 0;
    for (int i = 0; i < key.length(); i++) {
        for (int j = 0; j < key.length(); j++) {
            retVal += helperFun(key, i, j);
        }
    }
    return retVal;
}
```

NICHT effizient

Hashfunktion für Strings

Implementierung 1: Einfache Berücksichtigung von `key`

```
public int hashCode(String key) {  
    return key.length();  
}
```

Vorteil: sehr schnell
Nachteil: sehr viele Kollisionen!

Implementierung 2: Fortgeschrittene Berücksichtigung von `key`

```
public int hashCode(String key) {  
    int output = 0;  
    for(char c : key) {  
        output += (int)c;  
    }  
    return output;  
}
```

Vorteil: ziemlich schnell
Nachteil: einige Kollisionen

Hinweis: Alle Hashfunktion berechnen
am Ende % Arraygröße

Hashfunktion für Strings

Implementierung 3: Mehrere Aspekte von `key` berücksichtigt + Mathe!

```
public int hashCode(String key) {  
    int output = 1;  
    for (char c : key) {  
        int nextPrime = getNextPrime();  
        output *= Math.pow(nextPrime, (int)c);  
    }  
    return Math.pow(output, key.length());  
}
```

Vorteil: wenig Kollisionen

Nachteil: langsam, riesige Integer

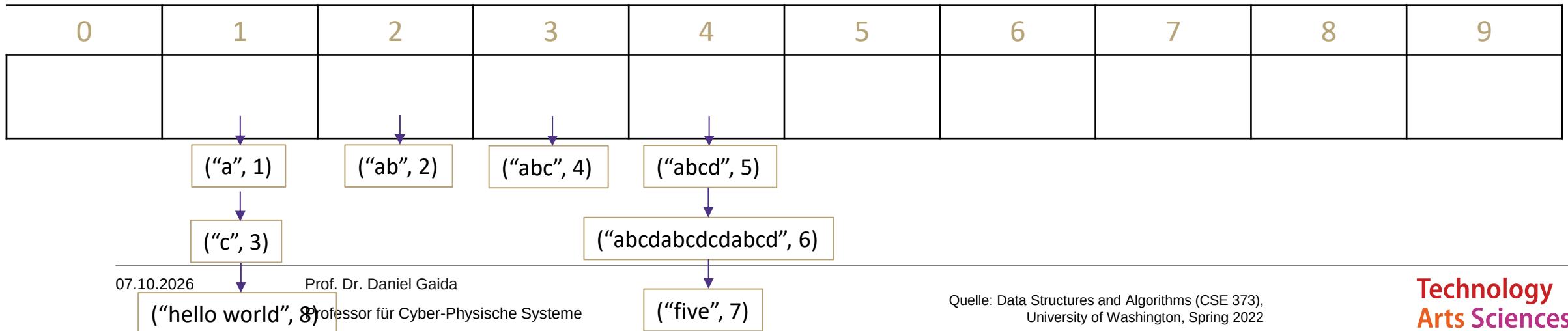
Hashfunktion für Strings - Übung

Befüllen Sie die Tabelle `data` mit einer Größe von 10 durch Hashing mit Verkettung. Nehmen Sie an, dass die Slots mit einer `LinkedList` implementiert sind. Verwenden Sie die folgende Hashfunktion:

```
public int hashCode(String key) {
    return key.length() % data.length;
}
```

Fügen Sie die folgenden Schlüssel-Wert-Paare ein. Wie sieht die Hashtabelle aus?

("a", 1) ("ab", 2) ("c", 3) ("abc", 4) ("abcd", 5) ("abcdabcdcdabcd", 6) ("five", 7) ("hello world", 8)



Größenänderung der Hashtabelle - Übung

Betrachten wir ein Array mit einer anfänglichen Kapazität von 5, dessen Kapazität jedoch vor $\lambda=1$ verdoppelt wird. Verwenden Sie die folgende Hashfunktion und Hashing mit Verkettung:

```
public int hashCode(String key) {
    return key.length() % data.length;
}
```

Fügen Sie die folgenden Schlüssel-Wert-Paare ein. Wie sieht die Hashtabelle aus?

("a", 1) ("ab", 2) ("cccc", 3) ("abcabc", 4) ("abcd", 5) ("abcdabcd", 6) ("five", 7) ("hello world", 8)

0	1	2	3	4
↓ ("cccc", 3)	↓ ("a", 1)	↓ ("ab", 2)		

↓
("abcabc", 4)

0	1	2	3	4	5	6	7	8	9
	↓ ("a", 1)	↓ ("ab", 2)		↓ ("abcd", 5)	↓ ("cccc", 3)	↓ ("abcabc", 4)		↓ ("abcdabcd", 6)	

↓
("hello world", 8)

↓
("five", 7)

Java und Hashfunktionen

Object class includes default functionality:

- equals
- hashCode

If you want to implement your own hashCode you should:

- Override BOTH hashCode() and equals()

If `a.equals(b)` is true then `a.hashCode() == b.hashCode()` MUST also be true

That requirement is part of the Object interface. Other people's code will assume you've followed this rule.

Java's HashMap (and HashSet) will assume you follow these rules and conventions for your custom objects if you want to use your custom objects as keys.

Fragen?

- Hashfunktionen für Strings
- Übungsblatt 4: Aufgabe 9 a

Strategien zum Umgang mit Hash-Kollisionen

Es gibt mehrere Strategien. In diesem Kurs werden wir die folgenden behandeln:

1. Hashing mit Verkettung

2. Offene Adressierung

- Lineare Sondierung
- Quadratische Sondierung
- Doppeltes Hashing

Umgang mit Kollisionen

Offene Adressierung

- Löst Kollisionen auf, indem ein anderer Slot zum Speichern eines Schlüssel-Wert-Paares gewählt wird, wenn der eigentliche Slot bereits belegt ist.

Typ 1: Lineare Sondierung

- Bei einer Kollision wird der nächste Slot geprüft, bis ein freier Slot gefunden wird.

```
int findFinalLocation(Key k)
{
    int naturalHash = this.getHash(k); int i=0;
    int index = naturalHash % ArraySize;
    while (index in use) {
        i++;
        index = (naturalHash + i) % ArraySize;
    }
    return index;
}
```

Offene Adressierung - Lineare Sondierung

Fügen Sie die folgenden Schlüssel in die Hashtabelle ein und verwenden Sie dabei eine Hashfunktion mit „% Arraygröße“ und lineares Sondieren, um Kollisionen aufzulösen. Annahme: `getHash()` liefert key zurück.

1, 5, 11, 7, 12, 17, 6, 25

0	1	2	3	4	5	6	7	8	9
	1	11	12		5	6	7	17	25

$\text{index} = 11 \% 10 = 1$

$i=1: (11 + 1) \% 10 = 2$

Zugehörige Werte werden ebenfalls in entsprechenden Index der Tabelle gespeichert.

Offene Adressierung - Lineare Sondierung

Fügen Sie die folgenden Schlüssel in die Hashtabelle ein und verwenden Sie dabei eine Hashfunktion mit „% Arraygröße“ und lineares Sondieren, um Kollisionen aufzulösen. Annahme: `getHash()` liefert `key` zurück.

38, 19, 8, 109, 10

0	1	2	3	4	5	6	7	8	9
8	109	10						38	19

Problem:

- Lineare Sondierung verursacht Clustering
- Clustering verursacht mehr Schleifendurchläufe beim Sondieren

Primäres Clustern

Wenn das Sondieren lange Ketten von belegten Slots in einer Hashtabelle verursacht

Offene Adressierung - Lineare Sondierung

Laufzeit

Wann ist die Laufzeit gut?

- Wenn wir auf leeren Slot treffen (oder es befindet sich ein leerer Slot in sehr kurzer Entfernung)

Wann ist die Laufzeit schlecht?

- Wenn wir auf einen primären Cluster treffen

Maximaler Belegungsfaktor?

- λ höchstens 1,0

Wann wird die Größe der Hashtabelle geändert?

- $\lambda \approx \frac{1}{2}$ ist eine gute Faustregel

Offene Adressierung - Quadratische Sondierung

Primäre Cluster werden durch die Auswahl des neuen Slots in der Nähe des eigentlichen Slots verursacht

Quadratische Sondierung:

- Anstatt i Slots nach dem eigentlichen Slot zu prüfen, prüfen Sie i^2 Slots nach dem eigentlichen Slot
- Parameter c_1 und c_2

```
int findFinalLocation(Key k)
    int naturalHash = this.getHash(k); int i = 0;
    int index = naturalHash % ArraySize;
    while (index in use) {
        i++;
        index = (naturalHash + c1*i + c2*i*i) % ArraySize;
    }
    return index;
```

Offene Adressierung - Quadratische Sondierung

Fügen Sie die folgenden Schlüssel in die Hashtabelle ein und verwenden Sie dabei eine Hashfunktion mit „% Arraygröße“ und quadratische Sondierung ($c_1=0$, $c_2=1$), um Kollisionen aufzulösen. Annahme: `getHash()` liefert `key` zurück.

89, 18, 49, 58, 79, 27, 9

index = $49 \% 10 = 9$
 $i=1: (49 + 1 * 1) \% 10 = 0$

0	1	2	3	4	5	6	7	8	9
49		58	79		9		27	18	89

index = $58 \% 10 = 8$
 $i=1: (58 + 1 * 1) \% 10 = 9$
 $i=2: (58 + 2 * 2) \% 10 = 2$

$79 \% 10 = 9$
 $i=1: (79 + 1 * 1) \% 10 = 0$
 $i=2: (79 + 2 * 2) \% 10 = 3$

$i=2: (9 + 2 * 2) \% 10 = 3$
 $i=3: (9 + 3 * 3) \% 10 = 8$
 $i=4: (9 + 4 * 4) \% 10 = 5$

Probleme:

Wenn $\lambda \geq \frac{1}{2}$, finden wir vielleicht nie einen freien Slot → Unendliche Schleife (Abbruch nach „Arraygröße“ Versuchen)!

Offene Adressierung - Quadratische Sondierung

Es kann passieren, dass wir nach Arraygröße Iterationen keinen freien Slot gefunden haben, obwohl noch freie Slots in der Hashtabelle existieren.

Bei quadratischer Sondierung ist nicht garantiert, dass alle möglichen Slots in der Hashtabelle überprüft werden.

Es gilt:

Wenn die Arraygröße eine Primzahl p ist, dann prüfen die ersten $p/2$ Sondierungen unterschiedliche Indizes.

Offene Adressierung - Quadratische Sondierung

Sekundäres Cluster

Fügen Sie die folgenden Schlüssel in die Hashtabelle ein und verwenden Sie dabei eine Hashfunktion mit „% Arraygröße“ und quadratische Sondierung ($c_1=0$, $c_2=1$), um Kollisionen aufzulösen

19, 39, 29, 9

0	1	2	3	4	5	6	7	8	9
39			29					9	19

Sekundäres Cluster

Bei Verwendung der quadratischen Sondierung muss manchmal die gleiche Abfolge von Slots untersucht werden, die nicht unbedingt nebeneinander liegen müssen.

Offene Adressierung - Quadratische Sondierung

Suche nach einem Schlüssel

Suchen Sie den Schlüssel 49 in der Hashtabelle und verwenden Sie dabei eine Hashfunktion mit „% Arraygröße“ und quadratische Sondierung ($c_1=0$, $c_2=1$). Wie viele Slots müssen Sie überprüfen, bis Sie feststellen, dass der Schlüssel 49 nicht in der Tabelle gespeichert ist?

0	1	2	3	4	5	6	7	8	9
39			29					9	19

Suche

Slots: 9 ($i=0$), 0 ($i=1$), 3 ($i=2$), 8 ($i=3$), 5 ($i=4$)

Slot 5 ist frei, deshalb existiert Schlüssel 49 nicht in Tabelle.

Offene Adressierung - Lineare und Quadratische Sondierung

- k = Schlüssel (Key)
- $h'(k)$ = der eigentliche Hashwert (naturalHash)
- $h(k, i)$ = Hashwert nach der Sondierung
- i = Iteration der Sondierung
- T = Arraygröße

Lineare Sondierung:

$$h(k, i) = (h'(k) + i) \bmod T$$

Quadratische Sondierung:

$$h(k, i) = (h'(k) + c_1 * i + c_2 * i^2) \bmod T$$

Fragen?

Behandelte Themen:

- Offene Adressierung zur Auflösung von Kollisionen:
 - Lineare Sondierung
 - Quadratische Sondierung

- Übungsblatt 4: Aufgabe 2 a, b und 8

Strategien zum Umgang mit Hash-Kollisionen

Es gibt mehrere Strategien. In diesem Kurs werden wir die folgenden behandeln:

1. Hashing mit Verkettung
2. Offene Adressierung
 - Lineare Sondierung
 - Quadratische Sondierung
 - **Doppeltes Hashing**

Offene Adressierung - Doppeltes Hashing

Beim Sondieren müssen wir immer wieder dieselben Indizes überprüfen - können wir stattdessen auch andere überprüfen?

Verwenden Sie eine zweite Hashfunktion!

$$\begin{aligned}h(k, i) &= (h_1(k) + i * h_2(k)) \bmod T \\ &= h_1(k) \bmod T + i * h_2(k) \bmod T\end{aligned}$$

← $h_2(k)$ muss einen Wert zurückliefern, der mit der Arraygröße T teilerfremd ist

```
int findFinalLocation(Key k)
{
    int naturalHash = this.getHash(k); int i = 0;
    int index = naturalHash % ArraySize;
    while (index in use) {
        i++;
        index = (naturalHash + i*jumpHash(k)) % ArraySize;
    }
    return index;
}
```

Offene Adressierung - Doppeltes Hashing

Wirksam, wenn $h_2(k)$ einen Wert liefert, der teilerfremd zur Arraygröße T ist

- Wenn T eine Potenz von 2 ist, muss $h_2(k)$ eine ungerade ganze Zahl zurückgeben.
- Wenn T eine Primzahl ist, soll $h_2(k)$ alles außer einem Vielfachen der Arraygröße zurückgeben

Offene Adressierung – Doppeltes Hashing

Suche nach einem Schlüssel

Suchen Sie den Schlüssel 49 in der Hashtabelle und verwenden Sie als erste Hashfunktion „% Arraygröße“ und als 2. Hashfunktion „3*key“. Wie viele Slots müssen Sie überprüfen, bis Sie feststellen, dass der Schlüssel 49 nicht in der Tabelle gespeichert ist? Annahme: `getHash()` liefert key zurück.

0	1	2	3	4	5	6	7	8	9
			29			39			19

Für $i = 1$:

$$(49 + 1 * 3 * 49) \% 10 =$$

$$= (49 + 147) \% 10 = 6$$

Suche

Slots: 9 ($i=0$), 6 ($i=1$), 3 ($i=2$), 0 ($i=3$)

Slot 0 ist frei, deshalb existiert Schlüssel 49 nicht in Tabelle.

Offene Adressierung - Größenänderung

Größenänderung funktioniert wie bei Hashing mit Verkettung:

- Anlegen einer neuen größeren Hashtabelle
- Evaluation der Hashfunktion für alle bereits eingefügten Paare und Einfügen der Paare in den richtigen Index

Wann wird Größe verändert?

- In Abhängigkeit des Belegungsfaktors λ und der Sondierungsstrategie
- Harte Grenzen:
 - Wenn $\lambda = 1$, `put()` schlägt bei linearer Sondierung mit einem neuen Schlüssel fehl
 - Wenn $\lambda > \frac{1}{2}$, `put()` könnte Sondierung bei der quadratischen Sondierung fehlschlagen, selbst mit einer primen Arraygröße
 - Und könnte früher fehlschlagen mit einer nicht-primen Arraygröße
 - Wenn $\lambda = 1$, `put()` schlägt doppeltes Hashing mit einem neuen Schlüssel fehl
 - Schlägt früher fehl, wenn Rückgabewert der 2. Hashfunktion nicht Teilerfremd zur Arraygröße ist

Offene Adressierung - Laufzeiten

Operation		Array w/ indices as keys
put(key, value)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$
get(key)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$
remove(key)	best	$\Theta(1)$
	In der Praxis	$\Theta(1+\lambda)$
	worst	$\Theta(n)$

- Worst Case: Wir müssen bei Sondierung $O(n)$ Indizes überprüfen (Cluster)
- In der Praxis: Rechtzeitig vergrößern bei $\lambda \approx \frac{1}{2}$

Fragen?

Behandelte Themen:

- Offene Adressierung zur Auflösung von Kollisionen:
 - Doppeltes Hashing

- Übungsblatt 4: Aufgabe 2 c

Löschen von Schlüssel-Wert-Paaren bei offener Adressierung

Übungsblatt 4:
Aufgabe 10

Lazy Deletion:

- Ein Schlüssel-Wert-Paar wird nicht vollständig gelöscht, sondern als gelöscht markiert. Gelöschte Stellen werden beim Einfügen als leer und bei einer Suche als belegt behandelt. Die gelöschten Stellen werden als tombstones bezeichnet. → Konstante Laufzeit
- Wenn bei erfolgreicher Suche ein tombstone betreten wurde, wird tombstone durch gefundenes Schlüssel-Wert-Paar ersetzt.
- Bei Größenänderung der Hashtabelle verschwinden alle tombstones.
- Bsp.: Quadratische Sondierung ($c_1=0$, $c_2=1$, % Arraygröße)
 - Lösche: 39; Suche: 29; Suche: 9; Einfügen: 49

0	1	2	3	4	5	6	7	8	9
								9	19

Zusammenfassung

1. Wählen Sie eine Hashfunktion, die:

- Kollisionen vermeidet
- Schlüssel-Wert-Paare gleichmäßig verteilt
- die Hash-Rechenkosten niedrig hält

2. Wählen Sie eine Kollisionsstrategie

- Hashing mit Verkettung
 - Verkettete Liste
 - AVL-Baum
- Offene Adressierung
 - Lineare Sondierung
 - Quadratische Sondierung
 - Doppeltes Hashing

Kein Clustering
Potenziell „kompakter“ (λ kann höher sein)

Die Vermeidung des Clustering kann knifflig sein
Weniger kompakt ($\lambda < \frac{1}{2}$ halten)
Array-Zugriffe sind in der Regel um einen konstanten Faktor schneller
als das Durchlaufen von verketteten Listen

Andere Hashing-Anwendungen

Wir verwenden Hashing für Hashtabellen, aber es gibt viele andere Einsatzmöglichkeiten! Hashing ist eine wirklich gute Methode, um aus beliebigen Daten eine kurze und eindeutige Zusammenfassung der Daten zu erstellen.

Caching

- Sie haben eine große Installationsdatei heruntergeladen und möchten wissen, ob eine neue Version verfügbar ist. Anstatt die gesamte Datei erneut herunterzuladen, vergleichen Sie den Hashwert Ihrer Datei mit dem Hashwert der Datei auf dem Server (Dateiverifizierung)

Beim Hashing werden die Daten „versteckt“, indem sie gehasht werden; dies kann für die Sicherheit genutzt werden

- Für die Überprüfung von Passwörtern: Die Speicherung von Passwörtern im Klartext ist unsicher. Daher werden Ihre Passwörter als Hash gespeichert (**kryptographische Hashfunktion** → Kollision statistisch praktisch ausgeschlossen).
- Digitale Signaturen

Other Hashing Applications

git hashes (“identification”)

- That crazy number that is attached to each of your commits
- SHA-1 hash incorporates the contents of your change, the name of the files and the lines of the files you changed

Ad Tracking

- track who has seen an ad if they saw it on a different device (if they saw it on their phone don't want to show it on their laptop)
- <https://panopticlick.eff.org> will show you what is being hashed about you

YouTube Content ID

- Do two files contain the same thing? Copyright infringement
- Change the files a bit!

Java's HashMap Implementation

default array capacity is 16

- Iteration over collection views requires time proportional to the "capacity" of the HashMap instance (the number of buckets) plus its size (the number of key-value mappings). Thus, it's very important not to set the initial capacity too high - Javadocs

resizes at load factor 0.75

- As a general rule, the default load factor (.75) offers a good tradeoff between time and space costs. Higher values decrease the space overhead but increase the lookup cost - Javadocs

uses separate-chaining collision resolution

- Initially uses LinkedLists as the buckets
- After 8 collisions across the table at the next resize the buckets will be created as balanced trees to reduce runtime of possible worst case scenario - [Javarevisited](#)

Ihr Werkzeugkasten bis hierhin...

Abstrakte Datentypen

- Liste - Flexibilität, einfaches Verschieben von Elementen innerhalb der Struktur
- Stapel (Stack) - optimiert für First-in-Last-out-Reihenfolge
- Warteschlange - optimiert für die Reihenfolge First-in-First-out.
- Tabelle - speichert Schlüssel und Wert bei jedem Eintrag

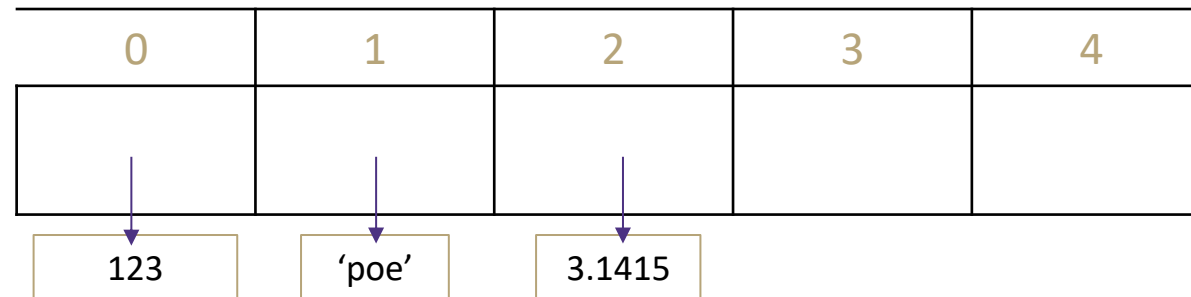
Implementierung von Datenstrukturen

- Array - leicht ein Element über Index zu suchen, schwer umzuordnen
- Verkettete Liste - schwer ein Element zu suchen, leicht umzuordnen
- Hashtabelle - konstante Zeit zum Suchen, keine Ordnung der Daten

Datenstrukturen in Python

Wie sind Listen in CPython implementiert?

- Listen sind als **Arrays** mit variabler Länge implementiert (ArrayList, s. Vorlesung 1, Folie 30). Die Implementierung verwendet ein zusammenhängendes Array von Referenzen auf andere Objekte.



```
liste = [123, 'poe', 3.1415]
liste[0]
liste.append('Dijkstra')
```

Datenstrukturen in Python

Wie sind Listen in CPython implementiert?

- Listen sind als **Arrays** mit variabler Länge implementiert (ArrayList, s. Vorlesung 1, Folie 30). Die Implementierung verwendet ein zusammenhängendes Array von Referenzen auf andere Objekte.
- Dies macht die Indizierung einer Liste `a[i]` zu einer Operation mit konstanter Laufzeit $O(1)$.
- Wenn Elemente angehängt oder eingefügt werden, wird die Größe des Arrays der Referenzen angepasst. Wenn das Array vergrößert werden muss, wird zusätzlicher Platz zugewiesen, damit bei den nächsten Einfügungen keine Größenänderung erforderlich ist.
- Die zum Einfügen eines Eintrags benötigte Zeit hängt von der Größe der Liste ab, genauer gesagt davon, wie viele Einträge sich rechts vom eingefügten Eintrag befinden ($O(n)$).

```
liste = [123, 'poe', 3.1415]  
liste[0]  
liste.append('Dijkstra')
```

Datenstrukturen in Python

Wie sind Tabellen in CPython implementiert?

- Tabellen (Dictionaries) in CPython sind als größenveränderbare Hashtabellen implementiert.
- Dictionaries funktionieren, indem für jeden im Dictionary gespeicherten Schlüssel mit Hilfe der eingebauten Funktion `hash()` ein Hashcode berechnet wird. Der Hashcode wird dann verwendet, um einen Speicherplatz in einem internen Array zu berechnen, in dem der Wert gespeichert werden soll.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
  
print(thisdict["brand"])
```

Nächste Vorlesung und Fragen

- Binäre Suchbäume und AVL-Baum
- Fragen?